

Smart-Kube: Energy-aware and Fair Kubernetes Job Scheduler using Deep Reinforcement Learning

Saeid Ghafouri

*School of Electronic Engineering
and Computer Science*

Queen Mary University of London
London, United Kingdom
s.ghafouri@qmul.ac.uk

Sina Abdipoor

*Department of Computer Science
Universiti Putra Malaysia*

43400 UPM Serdang, Malaysia
abdipoor.sina@student.upm.edu.my

Joseph Doyle

*School of Electronic Engineering
and Computer Science*

Queen Mary University of London
London, United Kingdom
j.doyle@qmul.ac.uk

Abstract—One of the most challenging problems in the popular orchestration framework Kubernetes is assigning sufficient resources to containers to operate at a required level while also avoiding excessive resource allocation which can delay other jobs in the cluster. A variety of heuristic approaches have been proposed to tackle this problem but these require considerable manual adjustments which can be laborious. Reinforcement learning approaches have been proposed to address this issue but these proposals do not consider the energy consumption of the cluster. This is an important component of the problem due to the commitments of large cloud operators to carbon neutrality. We have proposed a system called Smart-Kube to achieve a target utilization on nodes while maintaining energy consumption at a reasonable level. An experimental framework is designed on top of real-world Kubernetes clusters and real-world traces of container jobs are used to evaluate the framework. Experimental results show that Smart-Kube can approach the target utilization and reduce energy consumption in a variety of ways depending on the preferences of the cluster operator for a variety of cluster sizes.

Index Terms—Resource Management, Energy-Efficiency, Cloud Computing, Containers, Reinforcement Learning

I. INTRODUCTION

Recently containerization has become the dominant virtualization technology used in cloud computing platforms [1]. The management of diverse containers running in the cloud is a challenging problem and several orchestration frameworks have been developed to address these challenges. One of the most popular container orchestration frameworks is Kubernetes which is an open-source framework developed by Google.

One of the most important components of the orchestration framework is resource management which attempts to fairly distribute jobs to computing nodes in order to make sure that they function at a required level while also ensuring that the minimal number of physical nodes are utilized for serving containerized workloads. A variety of heuristic approaches for resource management in cloud platforms have been developed including fair scheduling [2], first fit [3] and simple packing strategies [4]. Several meta-heuristic approaches have also been proposed including genetic algorithms [5] and ant colony algorithms [6]. The success of these approaches is heavily dependent on the job scheduling algorithm utilized as the

resources required will depend on the number of jobs being directed to the Kubernetes nodes. Thus, the tuning of these algorithms can be a laborious process requiring considerable expertise. There have, however, been several proposals for using reinforcement learning in these platforms to avoid this with DeepRM [7] and Decima [8] being two notable proposals in this area.

While there has been considerable work in job scheduling for efficient job execution on container clusters, there has been little work in energy aware scheduling algorithms for Kubernetes clusters. Recently, however, large cloud service providers such as Microsoft and Amazon have increased their focus on reducing their environmental impact by committing to carbon neutrality by 2030 and 2040 respectively [9], [10]. Scheduling algorithms will likely need to consider the energy consumption of the container clusters due to increasing pressure from government organizations [11]. Thus, we propose the Smart-Kube system that considers both overall energy consumption and the load distribution of container clusters to provide good performance for its containers and a reduced environmental impact.

The *main contributions* of this research work are:

- We present the Smart-Kube system that integrates with Kubernetes to provide a scheduling service that distributes load evenly between container clusters while considering energy consumption. Smart-Kube models the problem of job scheduling while maintaining a reasonable energy consumption as an optimization problem and solves it using a reinforcement learning algorithm.
- We present a dataset of jobs traces that heavily utilizes containers for processing machine learning jobs. Our analysis shows that most Kubernetes pods are highly under utilized in terms of CPU and memory usage. This highlights the importance of scheduling algorithms that use actual resource usage as well as requested resources in scheduling algorithms. While the actual resource usage of jobs is not known at scheduling time Smart-Kube uses current job usage to schedule incoming jobs.
- We examine the performance of Smart-Kube with different reward weights on different cluster sizes to assess the trade-off between the two aforementioned objectives

as well as the scalability of the proposed solution. Our experiments show that users can prioritize their objects using the reward weights for all cluster sizes evaluated.

II. MOTIVATING SCENARIO

The increasing demand for scalable and efficient computing systems has resulted in the widespread adoption of cloud computing and container technologies, such as Kubernetes [1]. However, with the growing complexity of these systems, the management of their resources and allocation of jobs in a fair and efficient manner has become increasingly challenging [1].

Job scheduling in Kubernetes involves allocating computational resources, such as computing nodes, to different jobs running in the cluster. The primary objective of job scheduling is to allocate the resources in a way that balances resource utilization, job execution time, and cost efficiency while meeting the resource requirements of each job. This task becomes particularly challenging in large-scale Kubernetes clusters, where numerous jobs contend for the same resources. The scheduler must ensure that each job is assigned to an appropriate node, based on its resource requirements and the current state of the cluster. The scheduler must also continuously monitor the cluster, making adjustments as necessary to ensure that resources are allocated optimally and that jobs are running smoothly.

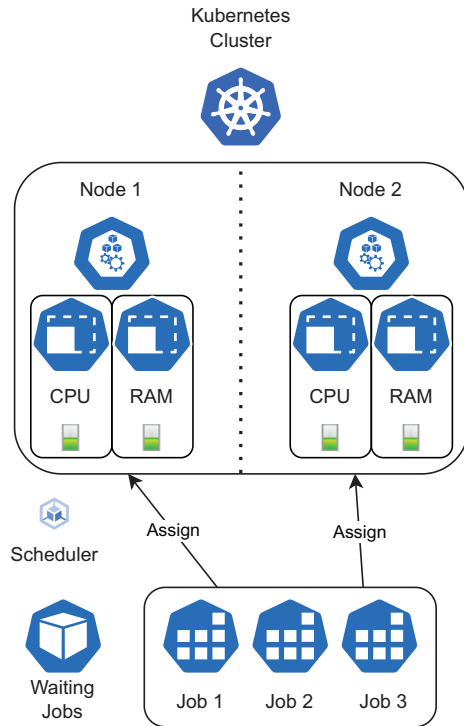


Fig. 1. Job Scheduling in a Kubernetes Cluster.

For instance, consider a scenario where a company has a Kubernetes cluster consisting of multiple nodes. Each node has limited amounts of computing resources such as CPU and

memory. The company has several batch jobs that run periodically, and each job requires a specific amount of resources. Some of these jobs require large computing resources, while others have more modest requirements. Figure 1 demonstrates a simplified example of a such scenario in a Kubernetes cluster. As illustrated in this figure, the aim of the scheduler is to distribute the tasks to the suitable nodes. In such cases, not only do we seek to handle the incoming jobs using the minimum number of nodes possible to achieve higher efficiency and minimize power usage but also to evenly distribute the workload among the nodes, which makes job failures less likely in the event of unforeseen spikes in resource usage.

III. RELATED WORKS

Reinforcement learning has been used for several job scheduling applications in cloud platforms. Mao *et al.* proposed DeepRM which uses reinforcement learning to schedule jobs. The resource requirements of the job are modeled as an image and this is used by the scheduling algorithm to minimize delays to the job completion time [7]. Mao *et al.* also presented Decima which uses reinforcement learning to schedule tasks with dependencies in Spark clusters. The tasks are modeled as a graph and graph convolutional networks as well as reinforcement learning are utilized to schedule the jobs [8]. Huang *et al.* proposed RLSK which utilizes reinforcement learning to balance resource utilization between container clusters [1]. Vrbaski *et al.* presented a system that uses reinforcement learning to schedule microservices while considering deadlines, cost and cluster utilization [12]. There have also been scheduling proposals on container platforms for niche applications such as HPKS proposed by Shi *et al.* which supports blockchain applications on container platforms [13]. Our work differs from these works in two principal ways. Firstly, these works do not consider the energy consumption of the clusters in their optimization formulations. Our work attempts to balance the performance of the jobs running on the cluster while also considering the energy consumption of the container clusters. Secondly, most of the results of these works are simulation based and do not experimentally evaluate these works on real cloud platforms. Our work has been integrated with Kubernetes and has been deployed on the Google Cloud Platform as part of our evaluation.

Minimizing the energy consumption of cloud computing components has been examined in numerous recent works. Séguéla *et al.* proposed a system for minimizing the energy consumption of data storage by adapting the data replication strategy [14]. Nordlund *et al.* proposed EALA which utilizes memory disaggregation on virtual machine platforms with a scheduling algorithm to reduce energy consumption [15]. Ghafouri *et al.* [16], [17] proposed an energy and mobility aware framework for Kubernetes edge clusters using reinforcement learning. Zhao *et al.* presented a geographical load balancing system for large clouds that shifts load based upon the availability of renewable energy [18]. Salmani *et al.* [19] presented an adaptive framework for minimizing cost and

maximizing accuracy on ML inference systems built on top of Kubernetes.

IV. PROBLEM FORMULATION

This section outlines the methodology employed in our study to address the Kubernetes job scheduling problem, including the representative structure, the optimization formulation, and the cost function.

A. Problem Representation

We represent this problem by modeling the environment states of the system and possible agent actions. Our representations are inspired by the previous work in this field [1], [7], [8], [16], [20].

- **States:** The states are represented by a one-dimensional vector that captures the current resource usage of each node as well as the characteristics of jobs waiting to be scheduled. The occupancy ratios of a node's resources indicate the overall resource usage of that node at a particular time. Similarly, the state of waiting jobs can be depicted by the required resource ratios from the node. To facilitate the representation of the states of nodes and jobs as an array of ratios between 0 and 1, we adopt the following assumptions:
 - All nodes have the same number (types) of resources;
 - Resources of the same type have consistent sizes across all nodes;
 - Time is assumed to be discrete and jobs arrive discretely in the system.

Indeed, these simplifications may not always align with real-world scenarios. However, they can significantly reduce the complexity and boost the convergence of the scheduler.

- **Actions:** An action can be characterized as the index of the node to which each job is assigned. The action represented by the index 0 can be used to temporarily postpone that job for a random time interval rather than attempting to schedule it immediately when the agents determine that the current system is not suitable for scheduling the job (e.g. due to high nodes workload). This allows the job to be delayed until a more optimal system state is reached.

Figure 2 illustrates a system in a hypothetical environment. In this example, the system consists of four nodes, identified as Nodes 1, 2, 3, and 4. Each of these nodes has two types of resources: CPU and RAM. We denote the utilization of resource r in node n by U_n^r , at the time $T = t$, the current resource utilization ratios for CPU and RAM in Node 1 are $U_{n_1}^{CPU}(t) = 10/100$ and $U_{n_1}^{RAM}(t) = 4/16$. Using these ratios to represent the state of nodes, the current state of Node 1 can be expressed as $S_{n_1}(t) = \{0.1, 0.25\}$. Likewise, the current states of Nodes 2, 3, and 4 can be represented as $S_{n_2}(t) = \{0.6, 0.5\}$, $S_{n_3}(t) = \{0.3, 0.125\}$, and $S_{n_4}(t) = \{0.75, 0.6875\}$, respectively. For job j , the requested utilization ratio of a resource r is denoted as U_j^r . As an example,

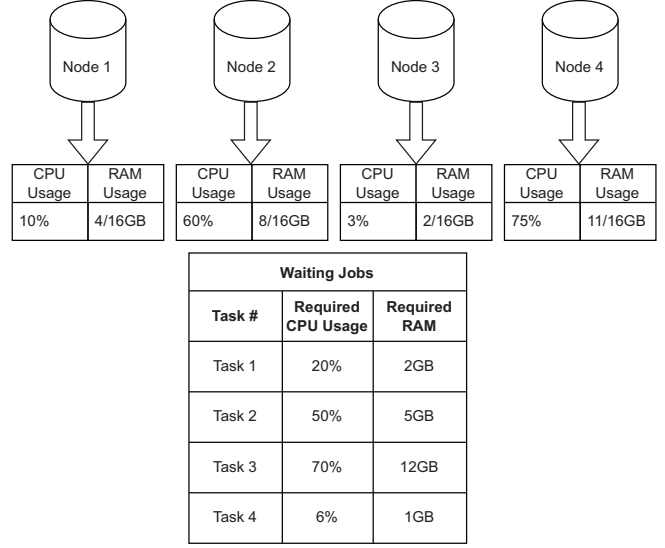


Fig. 2. Example of a Hypothetical Scheduling System.

in Job (Task) 1, $U_{j_1}^{CPU} = 0.2$ and $U_{j_1}^{RAM} = 0.125$. The state of the entire environment at the present time $T = t$, given the previously established cluster and job representations, can be defined as $S(t) = (\bigcup_{i=1}^4 S_{n_i}(t)) \cup (\bigcup_{i=1}^4 U_{j_i}^R)$ where R is the set containing all resources like RAM and CPU in the system. Therefore, $S(t)$ will be:

$$S(t) = \{\{0.1, 0.25\}, \{0.6, 0.5\}, \{0.3, 0.125\}, \{0.75, 0.6875\}, \{0.2, 0.125\}, \{0.5, 0.3125\}, \{0.7, 0.75\}, \{0.06, 0.0625\}\} \quad (1)$$

Assuming that the scheduler has assigned Job 1 to Node 2 and Job 2 to Node 1, the state of the environment at the subsequent time $T = t + 1$ will be:

$$S(t+1) = \{\{0.6, 0.5625\}, \{0.8, 0.625\}, \{0.3, 0.125\}, \{0.75, 0.6875\}, \{0.7, 0.75\}, \{0.06, 0.0625\}\} \quad (2)$$

B. Optimization Objectives

To steer the scheduler, we have specified two primary objectives:

- **Objective 1 ($G(t)$):** Minimizing the difference between actual utilization and a target utilization value;
- **Objective 2 ($P(t)$):** Minimizing the number of nodes in use.

Given a set of nodes with a total size of N and a set of resources with a total number of R resources, let n be a particular node and r be a specific resource. The current average resource utilization of the node n at time t ($U_n(t)$) can be formulated as:

$$U_n(t) = \frac{\sum_{r=1}^R U_n^r}{R} \quad (3)$$

Thus, the overall average utilization of a cluster at time t for all its nodes can be calculated as:

$$U(t) = \frac{\sum_{n=1}^N U_n(t)}{N} \quad (4)$$

By maximizing $U(t)$, it is ensured that at any given point in time, the resources in all nodes of the system are being utilized to their fullest capacity. Assume that in our previous example (refer to Figure 2) the scheduler assigns Jobs 1, 2, and 4 to Nodes 3, 1, and 4 at time t . In this scenario, Job 3 can no longer be assigned to any node as the required $U_{j3}^{TCPU} = 0.7$ and $U_{j3}^{RAM} = 0.75$ are not available in the cluster. This objective aims to minimize the incidence of such events. A potential reallocation would be to assign Jobs 1, 2, 3, and 4 to Nodes 2, 1, 3, and 4, respectively. This would result in higher overall utilization, and more significantly, it would enable the scheduling of all jobs at time t .

In larger systems, multiple allocations of jobs to nodes may yield the same overall utilization. However, some of these configurations may result in an uneven distribution of loads among the nodes, with some bearing a higher burden and others carrying a lighter workload. This imbalance is detrimental as workloads on these machines are more likely to encounter problems in the event of spikes in the workloads. The US Government has specified a maximum target utilization for physical machines of 75% as values exceeding this are likely to encounter issues [11]. $V(t)$ seeks to mitigate this issue by minimizing the variance between nodes' utilizations $U_n(t)$. The variance of resource utilization among all nodes in a cluster can be quantified using the following mathematical formulation, where μ represents the mean resource utilization across all nodes:

$$V(t) = \frac{\sum_{n=1}^N (U_n(t) - \mu)^2}{N} \quad (5)$$

Our second objective ($P(t)$) aspires to schedule all the jobs to the minimal number of nodes attainable. Doing so will enable us to completely power off the unutilized nodes, thus resulting in energy conservation and attaining a higher overall efficacy. We introduce the in-use node variable P_n at time t , which indicates whether a node is processing requests and whether it can be shut down to reduce costs. The variable is defined mathematically as:

$$P_n(t) = \begin{cases} 0, & \text{if } U_n^r = 0 \ \forall r \\ 1, & \text{if } \sum_{r=1}^R U_n^r > 0 \end{cases} \quad (6)$$

It is crucial to note that in the formulation of $P_n(t)$, especially in real-world applications, the 0 value refers to the node being at an idle stage, rather than indicating absolute 0 resource utilization. This means that a node can still be utilizing some resources, but not enough to be considered actively processing requests. In other words, the 0 value in

the formulation of $P_n(t)$ is used to indicate whether a node is idle and can be shut down or placed in a low power state to reduce costs, rather than indicating a node is not utilizing any resources at all.

The total in-use node count variable at time t , $P(t)$, can be calculated as the sum of the in-use node variables for all nodes as follows:

$$P(t) = \sum_{n=1}^N P_n(t) \quad (7)$$

This formulation allows us to evaluate the overall cost efficiency of the cluster by determining which nodes are idle and can be shut down to save costs.

C. Cost Function

We propose a cost-aware approach for fair job scheduling in Kubernetes clusters that is based on two key formulations: target utilization and minimum nodes in use. The target utilization formulation is used to ensure that resources are being used efficiently, while the minimum nodes in use formulation $P(t)$ is used to minimize the number of active nodes and thus reduce costs.

Target utilization, $G(t)$, is a key aspect of our proposed approach for fair job scheduling in Kubernetes clusters. It combines maximizing overall utilization, $U(t)$, and even load distribution function, $V(t)$, to reduce complexity in the scheduler's decision-making process, leading to more efficient resource usage and cost reduction for the cluster. Additionally, this combination of factors helps with the training process, as it needs to be calculated multiple times to reach convergence. We define the target utilization value as a hyperparameter that defines the maximum allowed load (percentage of resource utilization) in a cluster that aims to bring the nodes' utilization $U_n(t)$ as close as possible to this defined value, ensuring both high overall utilization and low utilization imbalance. Guidelines from the US Government indicate that the utilization level should be between 55% and 75% depending on the operator [11]. Target utilization at time t and given the target value of x can be formulated as:

$$G(t, x) = \frac{\sum_{n=1}^N (U_n(t) - x)^2}{N} \quad (8)$$

Given the target utilization $G(t, x)$ and the total in-use node count variable $P(t)$, we define the overall cost of a state at time t as follows, where w_1 and w_2 are the predefined weight values given to these two, respectively. The main objective of the scheduler is to minimize the following cost function:

$$C(t) = w_2 P(t) - w_1 G(t, x) \quad (9)$$

V. PROPOSED REINFORCEMENT LEARNING

The scheduling of jobs present in our problem can be viewed as a sequential decision-making problem that preserves the Markov property [21]. In such scenarios, deep reinforcement learning provides a suitable solution [22]. The objective

of deep reinforcement learning is for an agent to learn a policy, denoted as $\pi(s)$, which maximizes the discounted reward received from the environment. To achieve this, the value of the states is first computed. This value represents the expected sum of rewards, starting from the current state until reaching a terminal state, and is denoted as [23]:

$$v_\pi(s) = \mathbb{E}\pi[Rt + 1 + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots | S_t = s] \quad (10)$$

Maintaining all state values in a table is infeasible for problems with large state spaces. This is where a neural network is used, which can store all state values and update its weights at each time step of the reinforcement learning algorithm, based on the reward received from the environment.

In value-based reinforcement learning, the neural network receives the state of the environment as input and returns the state value as output. Subsequently, a maximization or greedy policy step is taken to select the appropriate action, based on the output state values received from the neural network. On the other hand, in policy gradient methods, the neural network returns the action directly as the output [23]. The overall structure of the value-based RL algorithms used is shown in Algorithm 1.

Algorithm 1 Deep Q-Network (DQN) Algorithm

```

1: Initialize the DQN with random weights
2: for  $episodes = 1, 2, \dots, M$  do
3:   Reset the environment to start a new episode
4:   Set the initial state  $s$ 
5:   for  $timesteps = 1, 2, \dots, T$  do
6:     Choose an action  $a$  using  $\epsilon$ -greedy exploration strategy
7:     Take the action  $a$  and observe the reward  $r$  and the new state  $s'$ 
8:     Store the transition  $(s, a, r, s')$  in the replay buffer
9:     Sample a random mini-batch of transitions from the replay buffer
10:    Compute the target Q-values for each transition:
11:      
$$\begin{cases} r & \text{episode terminates} \\ r + \gamma \max_{a'} Q(s', a'; \theta^-) & \text{otherwise} \end{cases}$$

12:    Update the DQN weights by minimizing the mean squared error loss:
13:      
$$\mathcal{L}(\theta) = \frac{1}{N} \sum_i (Q_{\text{target}}(s, a) - Q(s, a; \theta))^2$$

14:    Set the new state  $s \leftarrow s'$ 
15:    If the episode has ended, break the loop
16:  end for
17: end for

```

Agent In our experimentation, we utilized the Deep Q-Network Neural Network (DQN) as the agent. DQN is a model-free off-policy reinforcement learning algorithm [24]. It extends the Q-Learning algorithm by employing a deep neural network to estimate the Q-value function. The Q-value represents the expected total reward that the agent will receive from a specific state-action pair.

To enhance the learning process, DQN employs a replay buffer to store states, actions, rewards, and next states. This buffer allows the agent to learn from a diverse set of experiences, ultimately improving learning efficiency.

The training of the DQN algorithm involves minimizing the mean-squared error between the predicted Q-values and the actual Q-values. The training targets are computed using the Bellman equation, which updates the DQN Neural Network weights by minimizing the following loss function:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_i (Q_{\text{target}}(s, a) - Q(s, a; \theta))^2 \quad (11)$$

Here, s and a represent the current state and action, respectively. The immediate reward received after taking action a in state s is denoted as r . The discount factor γ in Line 11 accounts for the importance of future rewards, and s' and a' correspond to the next state and action. The target Q-value is computed as the sum of the immediate reward and the discounted maximum Q-value of the next state.

To balance exploration and exploitation, DQN utilizes an epsilon-greedy policy. This policy enables the agent to choose a random action with probability ϵ and the action with the maximum Q-value with probability $(1 - \epsilon)$. During training, the value of ϵ gradually decreases, shifting the agent's focus from exploration to exploitation as learning progresses.

State We base our representation of the environment on the state formulation explained in Section IV-A. To derive the current state of the system for passing to the DQN, we concatenate three arrays: an n -sized array $U = \{S_{n_1}, S_{n_2}, \dots, S_{n_n}\}$ indicating the current resource utilizations of nodes 1 to n , an array $J = \{S_{j_1}, S_{j_2}, \dots, S_{j_j}\}$ indicating the current requested resource utilizations of waiting jobs 1 to j , and an additional n -sized array $L = \{S_{n_1}, S_{n_2}, \dots, S_{n_n}\}$ representing the overall allocated resource to each job for all nodes 1 to n . Equation 1 presents the overall state of the environment in Figure 2. The final state that is fed to the RL agent will be the concatenation of these parts $S = U \cup J \cup L$.

Action As explained in Section IV-A, the action map is represented as an array with the size of the waiting jobs, denoted as $A = \{id_1, id_2, \dots, id_j\}$. Here, id_1 , id_2 , and id_j represent the node IDs to which jobs 1, 2, and j are assigned, respectively. To postpone the assignment of a job, we use the value 0.

Reward The reward at each time step is determined based on the objective function, as described by Equation 8. However, instead of attempting to minimize this objective function, we use its negative value as the reward function, which we aim to maximize. This is expressed mathematically as:

$$R(t) = -C(t) \quad (12)$$

Here, $R(t)$ represents the reward at time t , and $C(t)$ represents the cost at that time step.

VI. ARABESQUE AI DATASET

To emulate a real-world Kubernetes cluster as closely as possible we have collected and used a real-world trace from

Arabesque AI¹. The dataset contains a number of batch jobs, for training financial models encapsulated as containers. Each of these containers possesses a resource request for CPU and memory. It is important to note that this is user specified and not related to the actual resource usage of the container. In our own analysis of the dataset we observed that most of the Kubernetes pods are highly under utilized in terms of CPU and Memory usage. Inefficient resource requests on Kubernetes clusters will lead to poor scheduling decisions if the scheduler is configured to use resource requests rather than actual usage while most of the Kubernetes nodes are barely using their requested resources. In Figure 3 and 4 we have plotted the histogram of all running containers over a month on the Arabesque AI Kubernetes clusters based on the percentage of actual resource usage when compared with the requested resources. You can see that for both CPU and Memory most of the containers are using significantly less than their requested resources.

A commonly used metric for resource usage efficiency is slack [25]. It is defined as the fraction of requested resources that are not used over a specific time period, usually a relatively long period like a day. We have also calculated the slack values over the entire time period of the dataset. It was found to be 94% slack for memory and 68% for CPU. This highlights that considerable efficiencies can be achieved by considering cluster utilization instead of request sizes and powering off physical infrastructure that is not needed. While this does increase the risk of Out of Memory errors and CPU throttling the dataset shows that these events would be rare.

An example of the CPU and memory usage of a container is shown in Figure 5. From the figure, we can see that the usage is considerably lower than the request limit. It never exceeds 40% in the case of CPU and 20% in the case of memory. It is also interesting to note that usage is relatively stable over the lifetime of the container. This highlights the minimal risk in using resource usage as well as request limits in the scheduling algorithm. While this is of course only one container our analysis shows that the majority of containers show similar usage patterns. The ratio between the utilization and limit varies somewhat but large dynamic changes in resource usage were only found in a minority of the container traces. Thus, Smart-Kube uses resource usage as well as resource requests when scheduling to attempt to maximize potential energy savings.

VII. SYSTEM DESIGN

We have designed a complete end to end experimental setup. The implementation of the Kubernetes side is all done in real-world Google Cloud cluster.

Kubernetes clusters consist of a number of nodes. *Nodes* are conceptually equivalent to servers in more traditional computing clusters. It is possible for a node to be a bare metal server or virtual machine of any kind. From a user's perspective, there is no difference between a Kubernetes node deployed on bare

¹<https://www.arabesque.com/ai/>

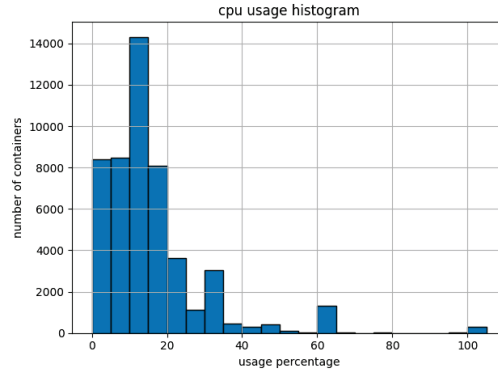


Fig. 3. The CPU resource usage to requested resource histogram of the containers at Arabesque over a month

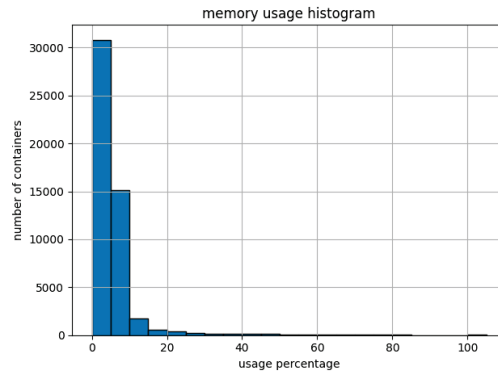


Fig. 4. The memory resource usage to requested resource histogram of the containers at Arabesque over a month

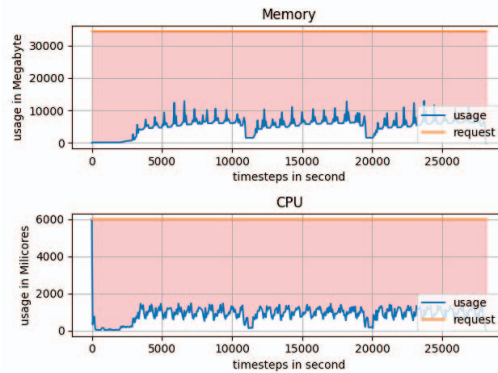


Fig. 5. An example of inefficient resource allocation for CPU and Memory

metal and a Kubernetes node deployed on a virtual machine. The minimum isolation level of the applications in Kubernetes is a container and Kubernetes possesses a further abstraction for representing containers with another entity called a *pod*. Pods are defined as a set of containers that share the same networking interface and are the smallest deployable unit in

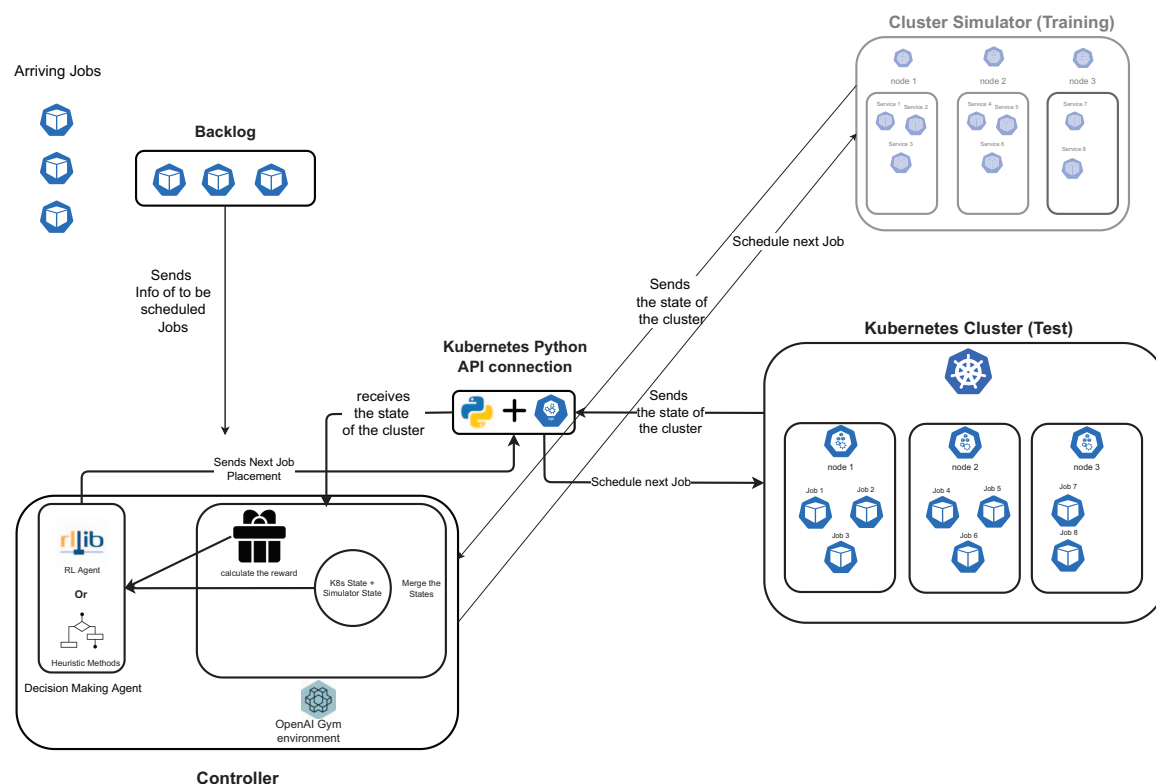


Fig. 6. The design of the system including of four parts (a) the controller in charge of making decisions about the placement of the jobs (b) The real world Kubernetes cluster (c) Kubernetes Python API connection to the Kubernetes cluster (d) Simulator of the real-world cluster

Kubernetes. All containers associated with a pod will always be co-located on the same cluster node. Pods are ephemeral objects and will be replaced and rescheduled during their lifetime. Kubernetes will use the concept of services to build a consistent representation of the networking information of the Kubernetes pods networking to enable access. The cluster is monitored and controlled using the Kubernetes API server. The API is essentially a REST server and there are numerous tools like the Kubernetes CLI named kubectl or other client APIs available in multiple languages. Our system utilizes the python client API to monitor and control the clusters.

Our definition of Jobs. Kubernetes schedules jobs as pods at the beginning of their lifetime and thus, we define jobs as the pods required to execute the workload of the Arabesque dataset described in Section VI. These pods will have diverse resource requirements and utilization profiles. For experimental purposes, we limit the definition of a job to a single pod containing one single container inside it with a Flask Python app for generating load on the container CPU and RAM. The Flask Python app is exposed to the outside world using Kubernetes' API.

Load Generation Module. To emulate artificial load on the jobs, we have used a model with two containerized Flask applications. One of them called the utilization server is deployed as a single Flask application in one arbitrary server

outside the cluster and the other one is an application running on all the pods. At each timestamp, the utilization server sends the resource usage of each application to them and the applications on the services put the load on the CPU and RAM of the application using a Linux tool named stress-ng.

Energy saving mechanism. One of the aims of our approach is to minimize the number of Kubernetes nodes available in the cluster. We have not implemented a mechanism for changing the power state of the physical machine as we are using virtual machines in our experiment. The Kubernetes pods could be drained from the cluster and the physical host could be powered off to achieve this for a real-world implementation.

Training. It is costly and time consuming to train on a physical cluster. Several timesteps are required to train the RL agents in the environment. Thus, it is impractical to execute the training on a physical cluster. We have implemented a simulator that mimics the dynamics of the real-world cluster during training to address this issue. The trained agent is then utilized outside the box in the real-world cluster for the experimental evaluation.

Complete System. As it can be seen in Figure 6, our design consists of three main parts. The first part is the Kubernetes cluster containing our Kubernetes nodes and jobs. The second is the cluster simulator which is connected to



Fig. 7. Normalized utilization for training under different priorities and cluster sizes

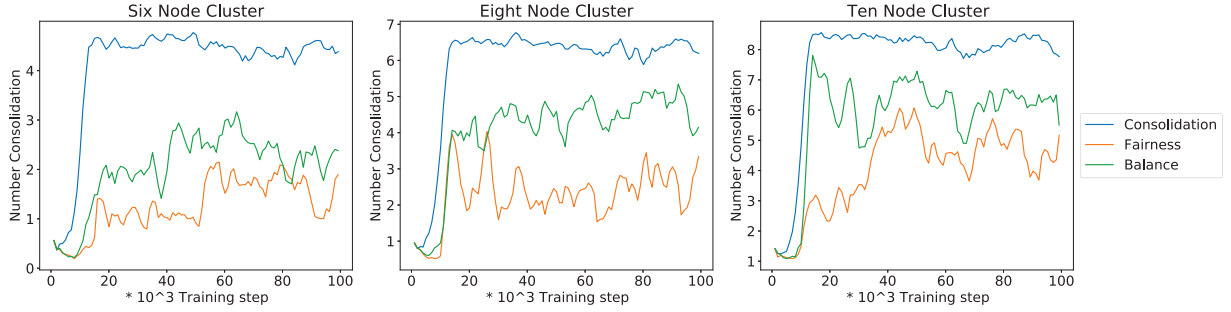


Fig. 8. Number of consolidated nodes for training under different priorities and cluster sizes

TABLE I
CONTENTS OF THE SYNTHETIC TRAINING DATASET

Number of containers	Memory Usage (MB)	CPU Usage (cores)
125	250	0.25
125	500	0.5
125	1000	1
125	2000	2
125	3000	3
125	4000	4
125	5000	5
125	6000	6

the cluster through a simulated connection. Essentially, this is a Python script assigning the jobs to the nodes. The placement of the nodes is sent to the controller. The controller receives information for both entities in a single OpenAI gym environment [26]. Rewards are then calculated based on the current observation of the environment. The RL agent will then decide upon the next placement of pods based on this information. rllib [27] has been used for the implementation of the RL agents. The Kubernetes controller will receive this placement from the RL agent and use this to schedule new jobs that arrive. An explanation of the framework interface is provided in the Appendix. All the codebase of this project is available at <https://doi.org/10.5281/zenodo.7693101>.

VIII. EXPERIMENTAL RESULTS

We used the Google GKE service to deploy our Kubernetes cluster. Due to our computational budget, we performed the experiments on a cluster that contained six, eight and ten

Kubernetes nodes. All the nodes were of n2-highcpu-8 type of the GKE platform with eight cores and 8 GB of Memory. The RL agents were trained on a synthetic dataset whose contents are summarized in Table I. We choose to use a synthetic dataset to allow us to evaluate the general performance of the algorithm for different datasets. For the testing experiments, jobs are selected randomly from the Arabesque dataset and a new job is scheduled every thirty seconds. A total of 2240 jobs were executed for each variation of the testing phase.

We evaluated the result in terms of the joint energy saving and utilization balancing objectives. We evaluate the performance of the algorithm using three different variations of the algorithm namely “Consolidation” which prioritizes the energy saving objective, “Fairness” which prioritizes the utilization balancing objective and “Balance” which prioritizes both rewards equally. For the “Consolidation” variation the weights are set to $(w_1 = 0, w_2 = 1)$. For the “Fairness” variation the weights were set to $(w_1 = 1, w_2 = 0)$. For the “Balance” variation the weights were set to $(w_1 = 0.5, w_2 = 0.5)$. We also evaluate the performance of the algorithm on different cluster sizes to evaluate how scale affects the performance of the algorithm. We utilize a six, eight and ten node cluster in our experiments.

During the training phase of the algorithm, we observed that all cluster sizes converge to values that prioritize the weights of the rewards. This is illustrated in Figure 7 and Figure 8. From these figures, we can see that “Consolidation” has the lowest normalized utilization reward and the highest number

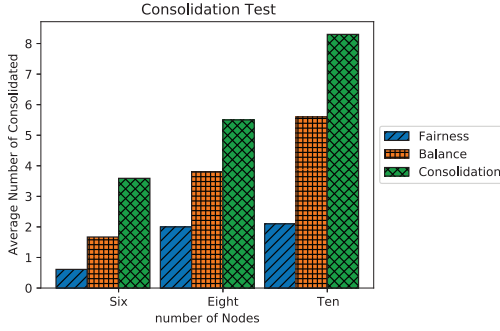


Fig. 9. Average server consolidation on test phase on Arabesque dataset.



Fig. 10. Average reward g on test phase on Arabesque dataset.

of consolidated nodes for all cluster sizes. We can also see that “Fairness” has the highest normalized utilization reward and lowest number of consolidated nodes for all cluster sizes while “Balance” has moderate rewards for both objectives for all cluster sizes.

For the test results, we also compared the results with the joint energy saving and target utilization objectives using the different variations of the algorithm for different cluster sizes. One key difference is the testing dataset is the Arabesque dataset described in Section VI. This allows us to evaluate the general performance of the algorithm for different datasets as there are considerable differences between the training dataset and the testing dataset. From Figure 9 and Figure 10 we can see that the trends found in the training dataset are also found when testing the algorithms with different datasets. “Consolidation” has the lowest normalized utilization reward and the highest number of consolidated nodes for all cluster sizes. “Fairness” has the highest normalized utilization reward and lowest number of consolidated nodes for all cluster sizes while “Balance” has moderate rewards for both objectives for all cluster sizes. This demonstrates that the algorithms can be used to achieve the desired objectives for different datasets.

IX. CONCLUSION AND FUTURE WORK

In this paper, we presented a deep reinforcement learning solution for reducing energy consumption in containerized clusters while maintaining a target utilization between the nodes in the cluster. Our results suggest that the trade-off between energy consumption and achieving target utilization can be achieved in a variety of ways by adjusting the reward weights while also improving performance in other areas. Although our proposed Smart-Kube job scheduler has shown promising results, there is still scope for future work to improve its performance and make it more applicable in real-world scenarios.

Utilizing *hierarchical and multi-agent RL* can be another direction to scale up and achieve better results in more complex environments. This is because these approaches can enable the handling of large numbers of jobs and more distributed data centers. *Re-scheduling of existing services* can be an alternative future research direction. By evaluating

the current state of the environment, Smart-Kube can predict the best time to re-schedule existing services. The selection of *optimal values for the parameters and hyperparameters* of the learning models was a challenging task that we had to tackle through our knowledge and experience. Future work can explore new techniques such as meta-heuristics, hyper-heuristics, or other approaches that can assist in setting the best parameter values for the algorithm.

REFERENCES

- [1] J. Huang, C. Xiao, and W. Wu, “Rlisk: a job scheduler for federated kubernetes clusters based on reinforcement learning,” in *2020 IEEE International Conference on Cloud Engineering (IC2E)*. IEEE, 2020, pp. 116–123.
- [2] A. Ghodsi, M. Zaharia, B. Hindman, A. Konwinski, S. Shenker, and I. Stoica, “Dominant resource fairness: Fair allocation of multiple resource types,” in *Nsdi*, vol. 11, no. 2011, 2011, pp. 24–24.
- [3] W. Song, Z. Xiao, Q. Chen, and H. Luo, “Adaptive resource provisioning for the cloud using online bin packing,” *IEEE Transactions on Computers*, vol. 63, no. 11, pp. 2647–2660, 2013.
- [4] R. Grandl, G. Ananthanarayanan, S. Kandula, S. Rao, and A. Akella, “Multi-resource packing for cluster schedulers,” *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 4, pp. 455–466, 2014.
- [5] F. Pezzella, G. Morganti, and G. Ciaschetti, “A genetic algorithm for the flexible job-shop scheduling problem,” *Computers & operations research*, vol. 35, no. 10, pp. 3202–3212, 2008.
- [6] P. Azad and N. J. Navimipour, “An energy-aware task scheduling in the cloud computing using a hybrid cultural and ant colony optimization algorithm,” *International Journal of Cloud Applications and Computing (IJCAC)*, vol. 7, no. 4, pp. 20–40, 2017.
- [7] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, “Resource management with deep reinforcement learning,” in *Proceedings of the 15th ACM Workshop on Hot Topics in Networks*, 2016, pp. 50–56.
- [8] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, “Learning scheduling algorithms for data processing clusters,” in *Proceedings of the ACM Special Interest Group on Data Communication*, 2019, pp. 270–288.
- [9] Microsoft, <https://blogs.microsoft.com/blog/2020/01/16/microsoft-will-be-carbon-negative-by-2030/>, accessed June 2020.
- [10] Amazon, <https://sustainability.aboutamazon.com/>, accessed June 2020.
- [11] A. Shehabi, S. Smith, D. Sartor, R. Brown, M. Herrlin, J. Koomey, E. Masanet, N. Horner, I. Azevedo, and W. Lintner, “United states data center energy usage report,” Lawrence Berkeley National Lab.(LBNL), Berkeley, CA (United States), Tech. Rep., 2016.
- [12] M. Vrbaski, M. Bolic, and S. Majumdar, “Scn-drl: Scheduler for large-scale critical notification applications based on deep reinforcement learning,” in *2022 9th International Conference on Future Internet of Things and Cloud (FiCloud)*. IEEE, 2022, pp. 1–9.

- [13] Z. Shi, C. Jiang, L. Jiang, and X. Liu, "Hpk: High performance kubernetes scheduling for dynamic blockchain workloads in cloud computing," in *2021 IEEE 14th International Conference on Cloud Computing (CLOUD)*. IEEE, 2021, pp. 456–466.
- [14] M. Séguéla, R. Mokadem, and J.-M. Pierson, "Dynamic energy and expenditure aware data replication strategy," in *2022 IEEE 15th International Conference on Cloud Computing (CLOUD)*. IEEE, 2022, pp. 97–102.
- [15] N. Nordlund, V. Vassiliadis, M. Gazzetti, D. Syrivelis, and L. Tassioulas, "Energy-aware learning agent (eala) for disaggregated cloud scheduling," in *2021 IEEE 14th International Conference on Cloud Computing (CLOUD)*. IEEE, 2021, pp. 578–583.
- [16] S. Ghafouri, A. A. Saleh-Bigdeli, and J. Doyle, "Consolidation of services in mobile edge clouds using a learning-based framework," in *2020 IEEE World Congress on Services (SERVICES)*. IEEE, 2020, pp. 116–121.
- [17] S. Ghafouri, A. Karami, D. B. Bakhtiarvan, A. S. Bigdeli, S. S. Gill, and J. Doyle, "Mobile-kube: Mobility-aware and energy-efficient service orchestration on kubernetes edge servers," in *2022 IEEE/ACM 15th International Conference on Utility and Cloud Computing (UCC)*, 2022, pp. 82–91.
- [18] J. Zhao, M. A. Rodríguez, and R. Buyya, "A deep reinforcement learning approach to resource management in hybrid clouds harnessing renewable energy and task scheduling," in *2021 IEEE 14th International Conference on Cloud Computing (CLOUD)*. IEEE, 2021, pp. 240–249.
- [19] M. Salmani, S. Ghafouri, A. Sanaee, K. Razavi, M. Mühlhäuser, J. Doyle, P. Jamshidi, and M. Sharifi, "Reconciling high accuracy, cost-efficiency, and low latency of inference serving systems," in *Proceedings of the 3rd Workshop on Machine Learning and Systems*, 2023, pp. 78–86.
- [20] S. Li, L. Wang, W. Wang, Y. Yu, and B. Li, "George: Learning to place long-lived containers in large clusters with operation constraints," in *Proceedings of the ACM Symposium on Cloud Computing*, 2021, pp. 258–272.
- [21] Z. Tang, X. Zhou, F. Zhang, W. Jia, and W. Zhao, "Migration modeling and learning algorithms for containers in fog computing," *IEEE Transactions on Services Computing*, vol. 12, no. 5, pp. 712–725, 2018.
- [22] H. Mao, P. Negi, A. Narayan, H. Wang, J. Yang, H. Wang, R. Marcus, M. K. Shirkoobi, S. He, V. Nathan *et al.*, "Park: An open platform for learning-augmented computer systems," in *Advances in Neural Information Processing Systems*, 2019, pp. 2494–2506.
- [23] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.
- [24] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing atari with deep reinforcement learning," *arXiv preprint arXiv:1312.5602*, 2013.
- [25] K. Rzaadca, P. Findeisen, J. Swiderski, P. Zych, P. Broniek, J. Kusmirek, P. Nowak, B. Strack, P. Witusowski, S. Hand *et al.*, "Autopilot: workload autoscaling at google," in *Proceedings of the Fifteenth European Conference on Computer Systems*, 2020, pp. 1–16.
- [26] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "Openai gym," *arXiv preprint arXiv:1606.01540*, 2016.
- [27] E. Liang, R. Liaw, R. Nishihara, P. Moritz, R. Fox, K. Goldberg, J. Gonzalez, M. Jordan, and I. Stoica, "Rllib: Abstractions for distributed reinforcement learning," in *International Conference on Machine Learning*. PMLR, 2018, pp. 3053–3062.
- [28] L. Espeholt, H. Soyer, R. Munos, K. Simonyan, V. Mnih, T. Ward, Y. Doron, V. Firoiu, T. Harley, I. Dunning *et al.*, "Impala: Scalable distributed deep-rl with importance weighted actor-learner architectures," in *International Conference on Machine Learning*. PMLR, 2018, pp. 1407–1416.
- [29] "Kubernetes Python Client," <https://github.com/kubernetes-client/python>.

X. APPENDIX

Smart-Kube is implemented on top of Kubernetes which is the most commonly used container orchestration framework. It seamlessly integrates with reinforcement learning algorithms through the OpenAI Gym interface [26]. Two modes of experimental setting have been implemented in Smart-Kube in

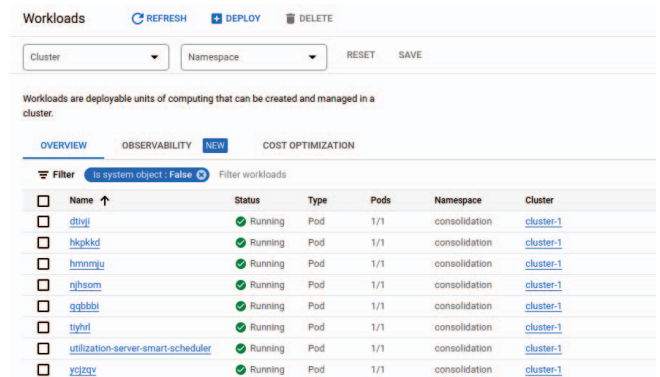
order to facilitate an easy-to-use interface for both simulated and real-world experiments.

```
# train.py
...
@click.command()
@click.option('--type-env', required=True,
              type=click.Choice(['sim-scheduler',
                                'kube-scheduler']),
              default='sim-scheduler')
...
```

The reinforcement learning experiments are automated through JSON configuration files carrying information about the Kubernetes cluster, the reinforcement learning agent, and experimental settings.

```
# config.json
{
  "env_config_base": {
    # Environment config (Experimental
    # setting)
  },
  "kube": {
    # Kubernetes cluster config
  },
  "run_or_experiment": "<RL algorithm name>",
  "learn_config": {
    # RL algorithm configuration
  },
}
```

In this study, we leverage DQN as our chosen reinforcement learning approach. However, the integration with rllib [27] within Smart-Kube enables the utilization of other types of RL algorithms, such as IMPALA [28]. To extract crucial telemetry data, such as CPU and memory usage, Smart-Kube employs the Google Cloud monitoring toolkit. Additionally, the scheduling framework is implemented using the Kubernetes Python client API [29]. Figure 11 demonstrates a typical run of Smart-Kube on a Google Cloud cluster in real-world mode, highlighting the practical application of our framework.



The screenshot shows the 'Workloads' section of the Smart-Kube interface. It includes a table with columns: Name, Status, Type, Pods, Namespace, and Cluster. All pods are in a 'Running' status. The pods are named with a prefix 'dtrvj' followed by a unique identifier. The namespace is 'consolidation' and the cluster is 'cluster-1'.

Name	Status	Type	Pods	Namespace	Cluster
dtrvj-...	Running	Pod	1/1	consolidation	cluster-1
dtrvj-...	Running	Pod	1/1	consolidation	cluster-1
dtrvj-...	Running	Pod	1/1	consolidation	cluster-1
dtrvj-...	Running	Pod	1/1	consolidation	cluster-1
dtrvj-...	Running	Pod	1/1	consolidation	cluster-1
dtrvj-...	Running	Pod	1/1	consolidation	cluster-1
dtrvj-...	Running	Pod	1/1	consolidation	cluster-1
dtrvj-...	Running	Pod	1/1	consolidation	cluster-1
dtrvj-...	Running	Pod	1/1	consolidation	cluster-1
dtrvj-...	Running	Pod	1/1	consolidation	cluster-1

Fig. 11. A run of Smart-Kube on top of a two-node Kubernetes cluster